

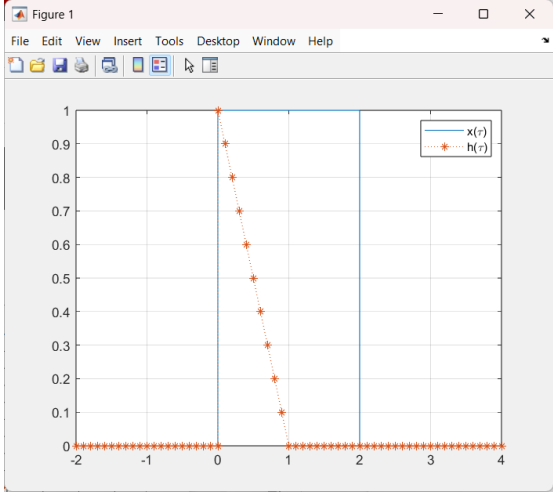
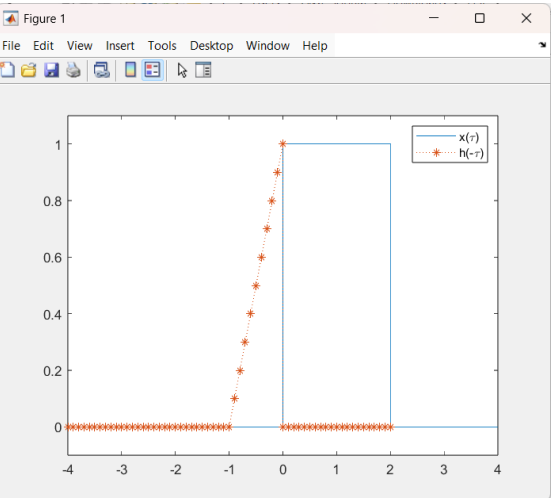
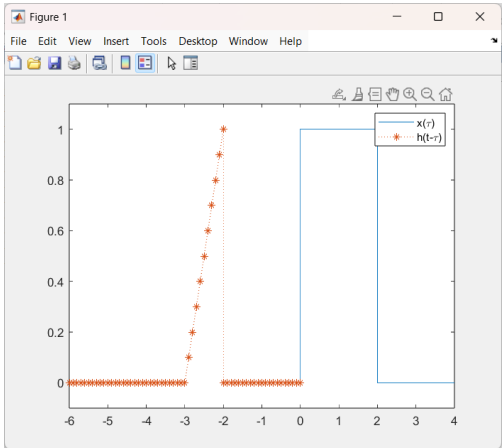


Name:	Lyka Sophia S. Jardeleza	Date:	09/18/2025
Program:	BSCpE	Section:	A461

A. Guided Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the MATLAB code.)

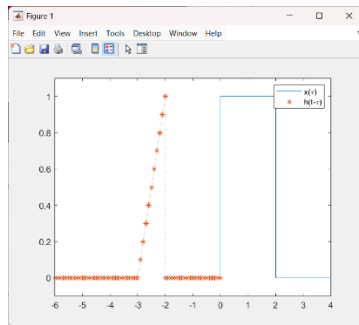
Code and Execute all the steps in Example 1

Photo/s of MATLAB output	Photos of MATLAB Codes
Step 1) 	Step 1) <pre> Editor - C:\Users\Lyka_Sophia\Downloads\DSP\Lab5_Exer1.m Lab5_Exer1.m 1 %Step 1 2 tx1=-2:0.1:0; 3 tx2=0:0.1:2; 4 tx3=2:0.1:4; 5 tx=[tx1 tx2 tx3]; 6 7 x1=zeros(size(tx1)); 8 x2=ones(size(tx2)); 9 x3=zeros(size(tx3)); 10 x=[x1 x2 x3]; 11 12 th1=-2:.1:0; 13 th2=0:.1:1; 14 th3=1:.1:4; 15 th=[th1 th2 th3]; 16 17 h1=zeros(size(th1)); 18 h2=1-th2; 19 h3=zeros(size(th3)); 20 h=[h1 h2 h3]; 21 22 plot(tx,x,th,h,'*') 23 legend('x(\tau)','h(\tau)') 24 grid </pre>
Step 2) 	Step 2) <pre> Editor - C:\Users\Lyka_Sophia\Downloads\DSP\Lab5_Exer1.m Lab5_Exer1.m 25 26 27 %Step 2 28 plot(tx,x,-th,h,'*') 29 legend('x(\tau)','h(-\tau)') 30 ylim([-0.1 1.1]) 31 </pre>
Step 3) 	Step 3) <pre> Editor - C:\Users\Lyka_Sophia\Downloads\DSP\Lab5_Exer1.m Lab5_Exer1.m 31 32 %Step 3 33 t=-2; 34 plot(tx,x,-th+t,h,'*') 35 ylim([-0.1 1.1]) 36 legend('x(\tau)','h(t-\tau)') </pre>

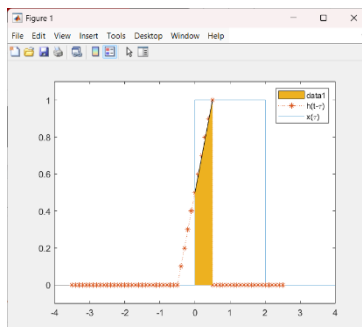


Step 4)

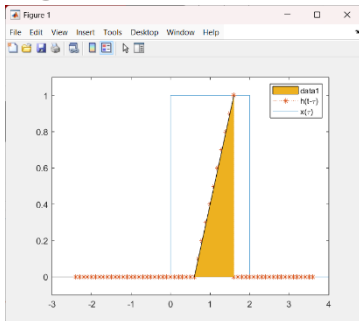
- Stage 1



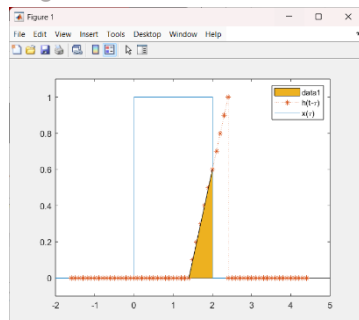
- Stage 2



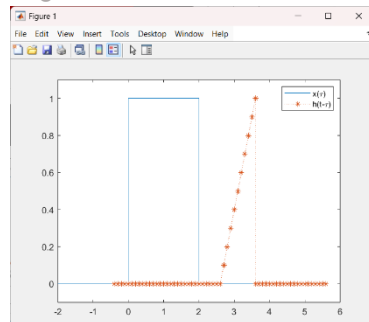
- Stage 3



- Stage 4



- Stage 5



Step 4)

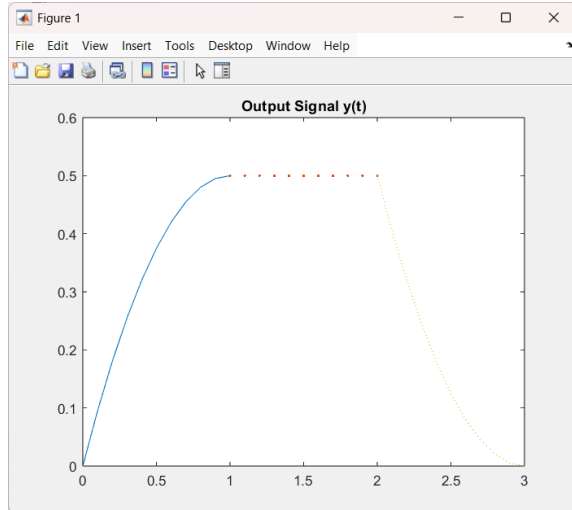
```

Editor - C:\Users\Lyka Sophia\Downloads\DSP\Lab5_Exer1.m
Lab5_Exer1.m
38 %Step 4
39 %First Stage: Zero Overlap
40 t=-2;
41 plot(tx,x,-th+t,h,'*')
42 ylim([-1 1.1])
43 legend('x(\tau)', 'h(t-\tau)')
44
45 %Second Stage: Partial Overlap
46 t=0.5;
47 plot(tx,x,-th+t,h,'*')
48 ylim([-1 1.1])
49 legend('x(\tau)', 'h(t-\tau)')
50 %the following code produces the shadowed area plot.
51 T=1;
52 r=0:1:T;
53 a=1/T*r+1-t/T;
54 hold on;
55 area(r,a);
56 hold off;
57
58 %Third Stage: Complete Overlap
59 t=1.6;
60 plot(tx,x,-th+t,h,'*')
61 ylim([-1 1.1])
62 legend('x(\tau)', 'h(t-\tau)')
63 r=t-T:1:t;
64 a=1/T*r+1-t/T;
65 hold on;
66 area(r,a);
67 hold off;
68
69 %Fourth Stage: Exit-Partial Overlap
70 t=2.4;
71 plot(tx,x,-th+t,h,'*')
72 ylim([-1 1.1])
73 legend('x(\tau)', 'h(t-\tau)')
74 r=t-T:1:2;
75 a=1/T*r+1-t/T;
76 hold on;
77 area(r,a);
78 hold off;
79
80 %Fifth Stage: Zero Overlap
81 t=3.6;
82 plot(tx,x,-th+t,h,'*')
83 ylim([-1 1.1])
84 legend('x(\tau)', 'h(t-\tau)')

```



Step 5)



Step 5)

```
Editor - C:\Users\Lyka Sophia\Downloads\DSP\Lab5_Exer1.m
Lab5_Exer1.m x +
86 % Step 5
87 t1=0:0.1:1;
88 t2=1:0.1:2;
89 t3=2:0.1:3;
90
91 y1=t1-(t1.^2)/2;
92 y2=0.5*ones(size(t2));
93 y3=0.5*((3-t3).^2);
94
95 plot(t1,y1,t2,y2,'.',t3,y3,':')
96 ylim([0 0.6])
97 title('Output Signal y(t)');
98
```

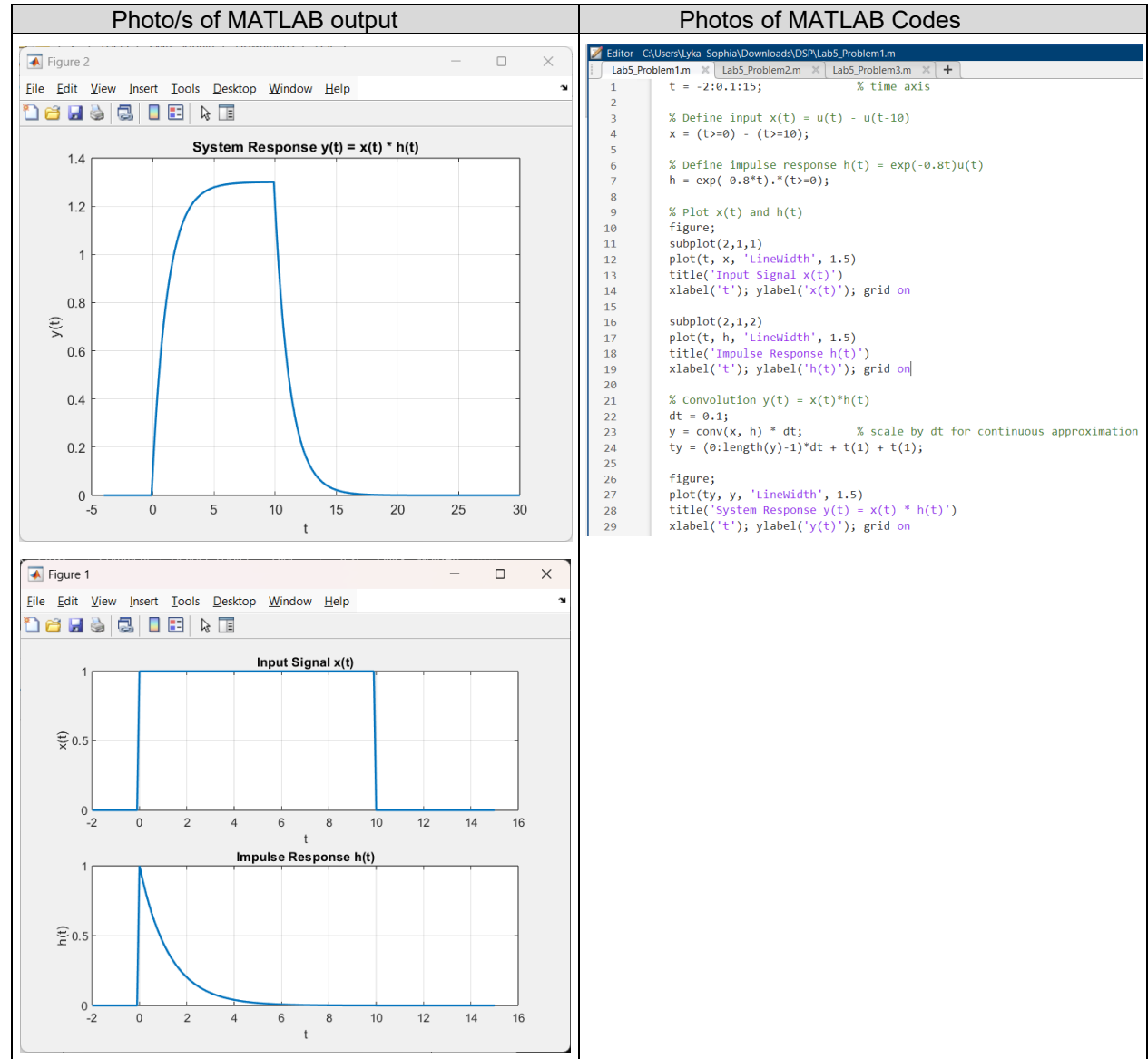


B. Problem Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the MATLAB code.)

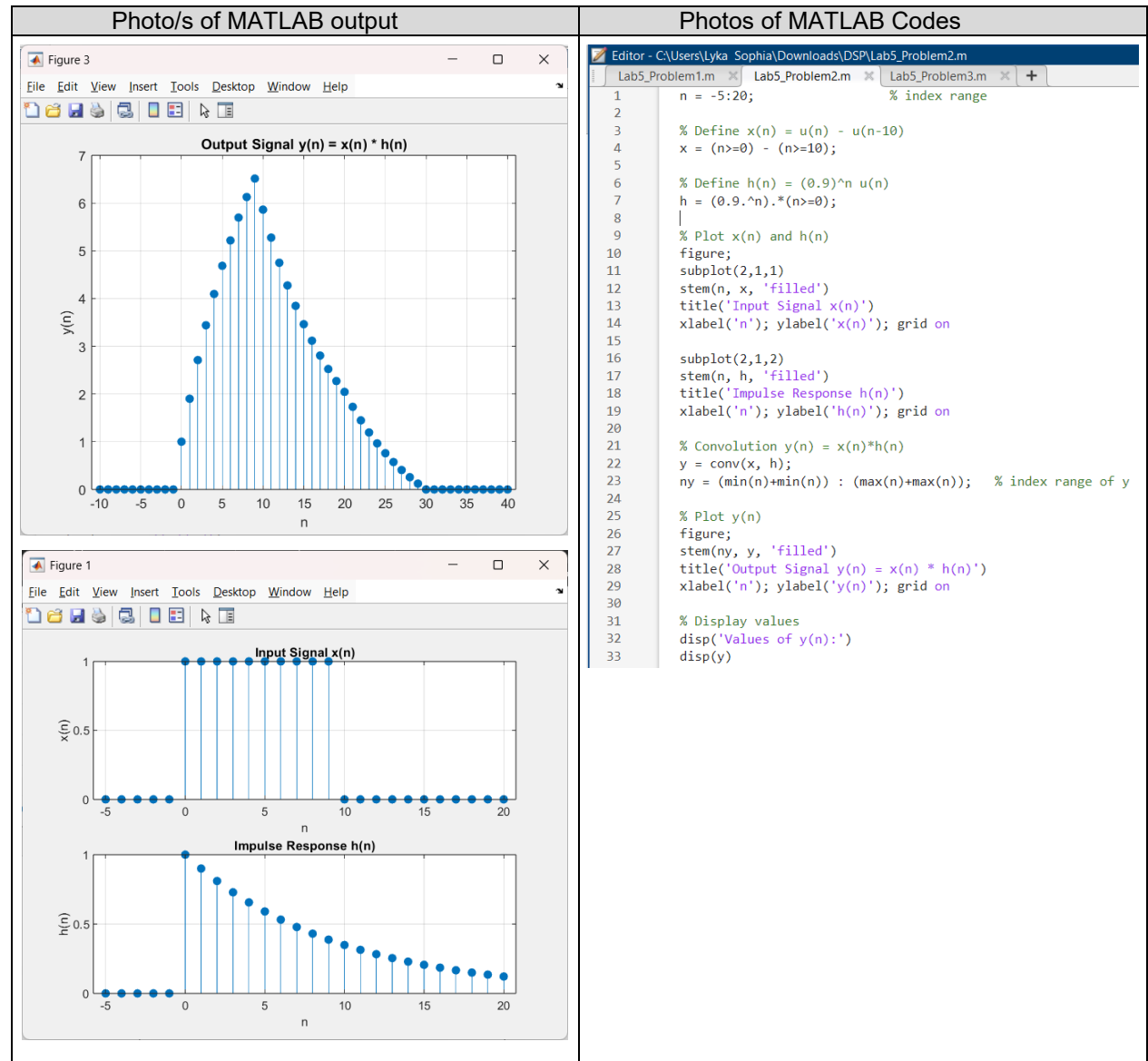
Solve the given problems in the Lab Guide (Exercise 1, 2, and 3) using MATLAB.

Problem 1:



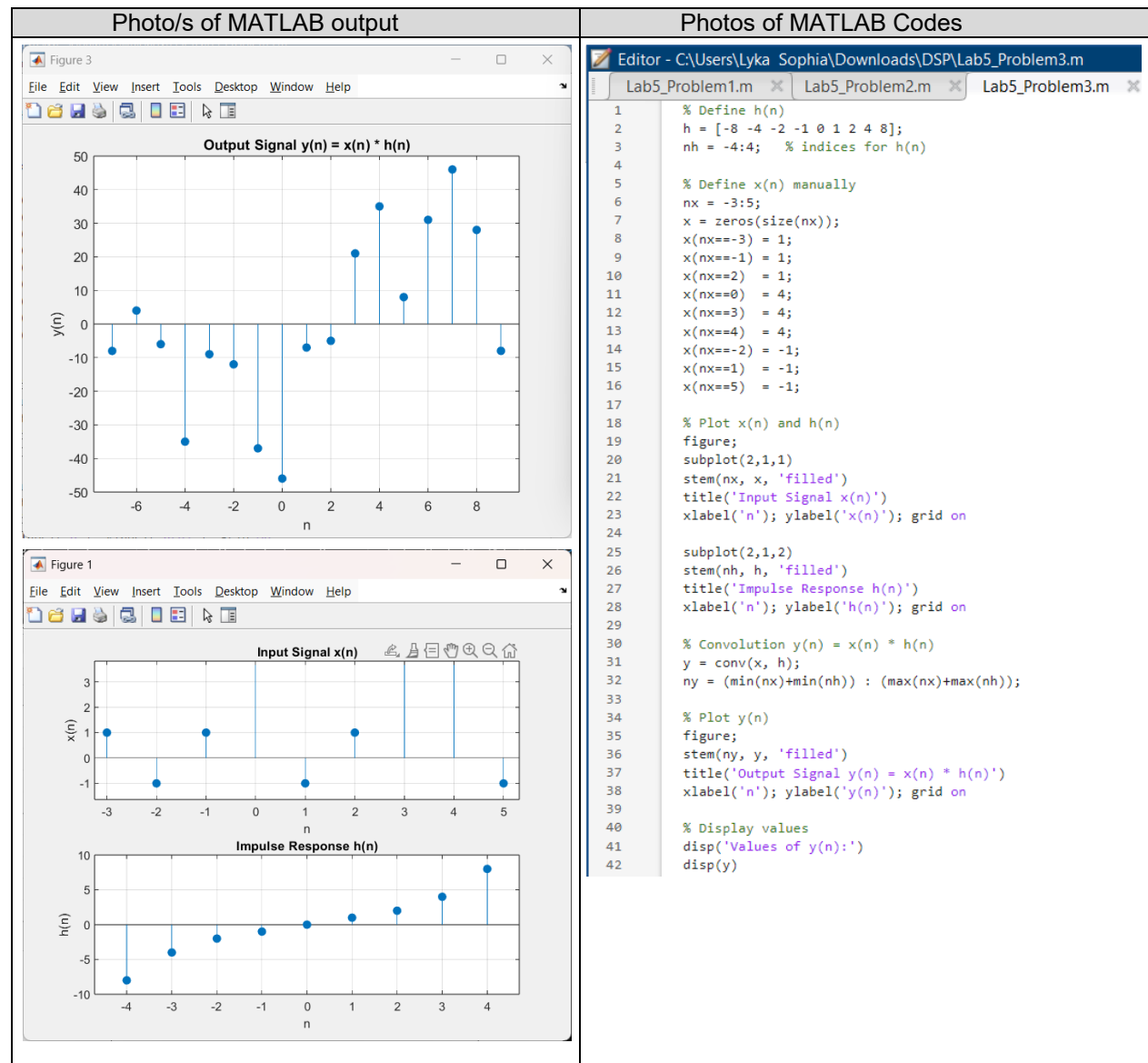


Problem 2:





Problem 3:





C. Findings ,Observations, and Comments:

Guiding Questions :

1. *What did you learn in the guided exercises? Did you learn something new?*
2. *What are your findings, observations, and comments in the guided exercises?*
3. *What are your findings, observations, and comments in the problem exercises?*

In the guided exercises, I learned how to apply convolution step by step using MATLAB. I realized that convolution is not just a formula but a process of sliding and overlapping signals to get the output. I also learned how to visualize the stages of convolution, which made the concept clearer compared to just solving it by hand.

I observed that the output signal really depends on how the input and impulse response overlap. The guided part showed me the importance of understanding each stage, from reflection to integration, because it makes the result easier to follow. MATLAB also helped confirm the theory since the plots matched the expected shapes.

In the problem exercises, I found out that convolution works the same idea for both continuous and discrete signals, but the implementation is different. For continuous signals, it uses integration, while for discrete it uses the conv function. I also observed that the length of the output is longer than the input, which makes sense since convolution extends the range of the signals. Overall, the problems showed me how convolution applies in real system responses.